

KPIT

Draft Documentation for LpsSaWeighApp and LpsSaJobMgrApp

1. Introduction

This document provides a comprehensive, structured explanation of the two primary applications:

- **Weigh Application** – LpsSaWeighApp
- **Job Manager Application** – LpsSaJobMgrApp

These applications are part of a larger **AIS-based distributed system** that operates on top of an **SCS communication framework**. The objective of this document is to enable a clear and complete understanding of:

- Functional responsibilities of each application
- Internal architecture and design patterns
- Communication mechanisms and data flow
- Runtime behavior and execution cycle
- Interdependencies between modules and external systems

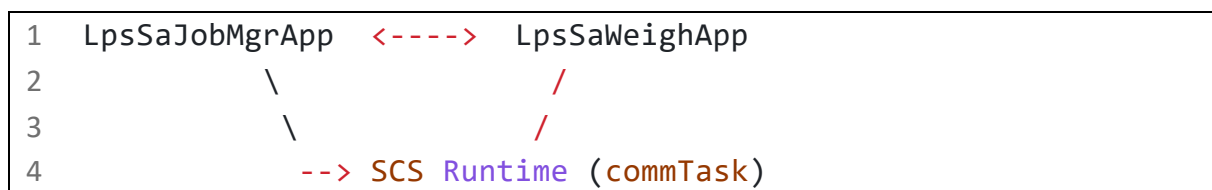
2. System Overview

The system consists of multiple applications running concurrently within an AIS runtime environment. These applications communicate via a centralized communication layer using SCS channels.

Key Characteristics:

- Distributed architecture with multiple always-running applications
- Communication via standardized interfaces using InterfaceDb
- Strong separation between business logic and communication logic
- Modular design enabling independent application execution

The primary interacting components are:



3. Application Responsibilities

3.1 LpsSaWeighApp

Location:

CPM-Loader-ais/apps/LpsSaWeighApp

Primary Responsibilities:

- Payload weight calculation
- Sensor data processing (PWM inputs, linkage, etc.)
- Calibration management
- Diagnostic state generation
- Publishing weight and system status

Core Functional Areas:

- Live weight computation
- Best bucket tracking
- Weight latching logic
- Calibration command handling
- Diagnostic state reporting

Dependencies:

- lps_weighing – core algorithm
- lps_cal – calibration logic
- Hardware inputs (PWM channels, machine data)

3.2 LpsSaJobMgrApp

Location:

CPM-Loader-ais/apps/LpsSaJobMgrApp

Primary Responsibilities:

- Job and pass management
- Payload tracking across loading cycles
- UI-facing command handling
- Operational decision-making
- Output actuation (e.g., horn control)

Core Functional Areas:

- Store / clear / minus-one operations
- Tip-off and standby state management
- Target weight tracking
- Lifecycle data tracking (payload, passes, counts)

Dependencies:

- lps_pass_tracker – job logic and pass tracking

4. Architectural Design

The system follows a **three-layer architecture**:

4.1 Application Layer

- Orchestrates execution flow
- Handles lifecycle (init, execute, shutdown)

4.2 Business Logic Layer

- Contains domain-specific algorithms
- Independent of communication mechanisms

4.3 Communication Layer

- Handles all input/output interactions
- Abstracts underlying SCS communication

```
1  +-----+
2  | Application Layer                      |
3  | (LpsSaWeighApp.cpp, LpsSaJobMgrApp.cpp) |
4  +-----+
5  +-----+
6  | Business Logic Layer                  |
7  | (lps_weighing, lps_cal, lps_pass_tracker) |
8  +-----+
9  +-----+
10 | Communication Layer                   |
11 | (SCS adapters, InterfaceDb, channels) |
12 +-----+
13
```

Important Property:

Business logic is isolated and does not directly depend on SCS transport.

5. Communication Framework

5.1 Core Mechanism

Applications communicate through **generated interface channels** using the following API pattern:

```
1  InterfaceDb::fetch("ChannelNameInput");  
2  channel->get(data);  
3  
4  InterfaceDb::fetch("ChannelNameOutput");  
5  channel->publish(data);
```

5.2 Communication Flow

Communication occurs through:

- **Input Channels** → Data received via get()
- **Output Channels** → Data sent via publish()

Data flows through:

```
1  Application  
2  → InterfaceDb  
3  → SCS Runtime  
4  → Other Applications
```

6. Channel-Based Interaction Model

6.1 Key Channels Between Applications

6.1.1 *JobMgr* → *Weigh*

- LpsSaWeighReqstChannel
- LpsSaJobMgrTxChannel

Used to:

- Send operational commands
- Provide job state and targets

6.1.2 *Weigh* → *JobMgr*

- LpsSaWeighRespChannel
- LpsSaWeighTxChannel

Used to:

- Provide responses to commands
- Share telemetry and weight data

6.1.3 *Bidirectional Channel*

- LpsSaJobMgrReqstChannel

Used by:

- UI → JobMgr
- Weigh → JobMgr (e.g., zero request)

6.2 Channel Data Categories

Channel Type	Purpose
Request Channels	Commands and control signals
Response Channels	Acknowledgments and success/failure
Tx Channels	Continuous state and telemetry

7. Execution Model

7.1 Common Lifecycle

Both applications follow the same lifecycle pattern:

1 Initialize → Receive → Process → Transmit → Repeat
--

7.2 Weigh App Execution Loop

1	<code>LpsSaWeighingScsRx()</code>	→ Read inputs
2	<code>LpsSaUpdt()</code>	→ Run weighing logic
3	<code>LpsSaWeighingScsTx()</code>	→ Publish outputs
4	Debug transmit	

7.3 JobMgr Execution Loop

1	<code>LpsSaJobMgrScsRx()</code>	→ Read inputs
2	<code>LpsSaJobMgrPtUpdate()</code>	→ Run job <code>logic</code> (weigh_mode)
3	<code>LpsSaJobMgrScsTx()</code>	→ Publish outputs
4	Debug transmit	

8. Internal Processing Flow

8.1 Weigh App Internal Flow

```
1 Request Channel
2 → SCS Adapter (set flags)
3 → Weigh Algorithm (lps_weighing)
4 → Response/Status Generation
5 → Publish via channels
```

Key observation:

- Requests are converted into **internal flags**
- Business logic operates independently of channel structure

8.2 JobMgr Internal Flow

```
1 SCS Inputs
2 → Input Data Structures
3 → weigh_mode() (pass tracking logic)
4 → Output Structures
5 → SCS publish
```

Key observation:

- Decision-making is centralized in `weigh_mode()`
- Communication happens only before and after logic execution

9. End-to-End Interaction

9.1 Primary Control Flow

```
1 UI → JobMgr → Weigh → JobMgr
```

9.2 Detailed Flow

1. UI sends command → JobMgr
2. JobMgr processes logic
3. JobMgr sends command → Weigh
4. Weigh processes sensors and algorithms
5. Weigh sends status → JobMgr
6. JobMgr updates job state

10. External Dependencies

10.1 Input Sources

- PwmInputChannels → sensor signals
- SwitchInputScs → physical inputs
- AisJhm2TxChannel → calibration/display data
- Machine/DataLinkData → configuration

10.2 Output

- OutputChannel → horn and actuator control

11. System Runtime Context

The entire system runs under AIS with:

- scs → communication runtime process
- Multiple application tasks running concurrently
- Channels connecting all processes

```
1  scs (communication runtime)
2  ↑
3  | (via InterfaceDb)
4  |
5  +-----+      +-----+
6  | JobMgr | <-> | Weigh  |
7  +-----+      +-----+
```

12. Architectural Characteristics

12.1 Modularity

- Clearly separated layers
- Decoupled communication and logic

12.2 Scalability

- Multiple applications communicate through common infrastructure

12.3 Maintainability

- Communication code isolated in SCS adapter files

12.4 Extensibility

- Possible to replace communication layer independently

13. Key Observations

1. Communication is entirely channel-driven
2. Business logic is independent of transport mechanism
3. Both applications follow identical execution patterns
4. Bidirectional interaction exists between Weigh and JobMgr
5. External inputs and outputs play a significant role

14. Summary

The system can be understood in the following simplified model:

- **Weigh App:** Performs measurement and calculation
- **JobMgr App:** Makes decisions based on those measurements
- **SCS Channels:** Serve as the communication bridge between them

Overall system behavior:

1 Sense (Weigh) → Decide (JobMgr) → Act → Repeat
--

Both applications are tightly coordinated but loosely coupled through a standardized communication framework, enabling maintainability and future portability.